

```

1  /* =====
2      kartendekor.js – Route, Massstabsleiste und Windrose
3
4      Reine Zeichenroutinen ohne Zugriff auf den Erzählzustand: sie bekommen
5      sichtbare Bbox, Kartenoffset, Alpha und beim Routenzug den Endindex als
6      Parameter. haversineMeter wohnt hier mit, weil zeichneMassstabsleiste sein
7      einziger Aufrufer ist.
8  ===== */
9
10 // --- Modulkapselung -----
11 // 10 von 13 Namen intern, 3 exportiert. Konvention: docs/architektur.md.
12 (function () {
13
14     // -----
15     // Massstabsleiste unten rechts, skaliert live mit der sichtbaren Bbox.
16     // Haversine bei mittlerer Breite reicht als Näherung für einen Ausschnitt.
17     // -----
18
19     function haversineMeter(lon1, lat1, lon2, lat2) {
20         const R = 6371000;
21         const toRad = d => d * Math.PI / 180;
22         const dLat = toRad(lat2 - lat1);
23         const dLon = toRad(lon2 - lon1);
24         const a = Math.sin(dLat / 2) ** 2 + Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) * Math.sin(dLon
25 / 2) ** 2;
26         return 2 * R * Math.asin(Math.sqrt(a));
27     }
28
29     // Rundwerte für die Balkenlänge in Metern, Übersicht bis Kapitel-Zoom.
30     const MASSSTAB_SCHRITTE = [10, 20, 25, 50, 100, 200, 250, 500, 1000, 2000, 2500, 5000, 10000, 20000,
31 25000, 50000, 100000];
32
33     function zeichneMassstabsleiste(bbox, offsetX, alphaMultiplier = 1) {
34         if (alphaMultiplier <= 0) return;
35         let mapPixelWidth = width - offsetX;
36         if (mapPixelWidth <= 0) return;
37         let midLat = (bbox.north + bbox.south) / 2;
38         let breiteMeter = haversineMeter(bbox.west, midLat, bbox.east, midLat);
39         let meterProPixel = breiteMeter / mapPixelWidth;
40         if (!isFinite(meterProPixel) || meterProPixel <= 0) return;
41
42         // Grösster "schöner" Wert, dessen Balken noch unter ~160px bleibt.
43         let ziel = MASSSTAB_SCHRITTE[0];
44         for (let schritt of MASSSTAB_SCHRITTE) {
45             if (schritt / meterProPixel <= 160) ziel = schritt;
46             else break;
47         }
48         let balkenBreite = ziel / meterProPixel;
49         let label = ziel >= 1000 ? `${ziel / 1000} km` : `${ziel} m`;
50
51         let randX = 40, randY = 36, tickHoehe = 6;
52         let x1 = width - randX - balkenBreite;
53         let x2 = width - randX;
54         let y = height - randY;
55
56         push();
57         stroke(26, 26, 26, 220 * alphaMultiplier);
58         strokeWeight(2);
59         line(x1, y, x2, y);
60         line(x1, y - tickHoehe, x1, y);
61         line(x2, y - tickHoehe, x2, y);
62         noStroke();
63         fill(26, 26, 26, 220 * alphaMultiplier);
64         textFont(SCHRIFT_SANS);
65         textStyle(NORMAL);
66         textSize(11);

```

```

65     textAlign(CENTER, BOTTOM);
66     drawingContext.fillText(label, (x1 + x2) / 2, y - tickHoehe - 4); // fillText: p5s text() bleibt
    bei Animation manchmal unsichtbar
67     pop();
68 }
69
70 // -----
71 // Windrose oben rechts. Winkel werden in Grad notiert (0 = Norden) und mit
72 // radians(winkel - 90) auf p5s Radiant-Modus und 0°=Osten umgerechnet.
73 //
74 // ACHTUNG derzeit ohne Aufrufer: der Aufruf in draw() (sketch.js) ist
75 // auskommentiert, weil oben rechts das Kreisgrafik-Icon steht. Funktion und
76 // Export bleiben absichtlich stehen.
77 // -----
78
79 function zeichneWindrose(x, y, groesse, alphaMultipller = 1) {
80     if (alphaMultipller <= 0) return;
81
82     const zinkgrau = '#9DA69D';
83     const kalksteinCreme = '#212B2E';
84     const schmiedeeisenSchwarz = '#9DA69D';
85     const cafeRot = '#212B2E';
86     const messingGold = '#212B2E';
87
88     // Zweigeteilter Zacken. winkel: 0 = Norden, im Uhrzeigersinn.
89     function zeichneZacke(winkel, radius, basisBreite, farbeLinks, farbeRechts) {
90         const w = radians(winkel - 90);
91         const spitzeX = radius * cos(w);
92         const spitzeY = radius * sin(w);
93         const basis1X = basisBreite * cos(w + HALF_PI);
94         const basis1Y = basisBreite * sin(w + HALF_PI);
95         const basis2X = basisBreite * cos(w - HALF_PI);
96         const basis2Y = basisBreite * sin(w - HALF_PI);
97
98
99         // Helle Kontur, sonst wirkt die Zacke auf der dunklen Startkarte
100        // einseitig – die dunkle Hälfte verschwindet.
101        stroke('#9DA69D');
102        strokeWeight(0.75);
103        fill(farbeLinks);
104        triangle(0, 0, spitzeX, spitzeY, basis1X, basis1Y);
105        fill(farbeRechts);
106        triangle(0, 0, spitzeX, spitzeY, basis2X, basis2Y);
107    }
108
109    push();
110    drawingContext.globalAlpha = alphaMultipller;
111    translate(x, y);
112
113    const rHaupt = groesse;
114    const rNeben = groesse * 0.6;
115
116    // Äussere Ringe
117    noStroke();
118    fill(226, 230, 225, 40); // #E2E6E1, sehr leicht
119    circle(0, 0, rHaupt * 2 + 20);
120    circle(0, 0, rHaupt * 2);
121
122    // Haupt-Zacken: Nord, Ost, Süd, West
123    const richtungenHaupt = [
124        { winkel: 0, farbeLinks: cafeRot, farbeRechts: schmiedeeisenSchwarz },
125        { winkel: 90, farbeLinks: kalksteinCreme, farbeRechts: schmiedeeisenSchwarz },
126        { winkel: 180, farbeLinks: kalksteinCreme, farbeRechts: schmiedeeisenSchwarz },
127        { winkel: 270, farbeLinks: kalksteinCreme, farbeRechts: schmiedeeisenSchwarz },
128    ];
129    const breite = groesse * 0.08;

```

```

130 richtungenHaupt.forEach(r => zeichneZacke(r.winkel, rHaupt, breite, r.farbeLinks, r.farbeRechts));
131
132 // Neben-Zacken: NO, SO, SW, NW
133 const richtungenNeben = [45, 135, 225, 315];
134 const breiteNeben = groesse * 0.05;
135 richtungenNeben.forEach(w => zeichneZacke(w, rNeben, breiteNeben, messingGold, zinkgrau));
136
137 // Zentrum
138 stroke(messingGold);
139 strokeWeight(1);
140 fill(kalksteinCreme);
141 circle(0, 0, groesse * 0.18);
142 noStroke();
143 fill(schmiedeeisenSchwarz);
144 circle(0, 0, groesse * 0.05);
145
146 // Eigene Farbe, weil die Zacken-Konstanten dafür zu hell sind.
147 // fillText direkt: p5s text() bleibt bei Animation manchmal unsichtbar.
148 function zeichneBeschriftung(label, x, y) {
149   drawingContext.fillText(label, x, y);
150 }
151
152 const beschriftungsFarbe = '#A4860A';
153 noStroke();
154 fill(beschriftungsFarbe);
155 textAlign(CENTER, CENTER);
156 textSize(groesse * 0.2);
157 textFont(SHRIFT_SANS);
158 textStyle(BOLD);
159 zeichneBeschriftung('N', 0, -rHaupt - 16);
160 zeichneBeschriftung('O', rHaupt + 16, 0);
161 zeichneBeschriftung('S', 0, rHaupt + 16);
162 zeichneBeschriftung('W', -rHaupt - 16, 0);
163
164 // Dieselbe -90-Ausrichtung wie die Zacken, sonst landet "NO" auf SO.
165 fill(beschriftungsFarbe);
166 textSize(groesse * 0.1);
167 const offsetNeben = rNeben + 14;
168 richtungenNeben.forEach((w, i) => {
169   const label = ['NO', 'SO', 'SW', 'NW'][i];
170   const a = radians(w - 90);
171   zeichneBeschriftung(label, offsetNeben * cos(a), offsetNeben * sin(a));
172 });
173
174 pop();
175 }
176
177
178
179 // -----
180 // Die Route: eine Farbe, nach hinten verblassend. Gezeichnet wird in einen
181 // eigenen Puffer, weil nur dort Deckkraft geschrieben statt gemischt wird.
182
183 // Jede Stufe wird DECKEND gezogen und überschreibt die vorherige; den Verlauf
184 // macht ein Waschgang davor (erase = destination-out).
185
186 // ACHTUNG nicht mit halbdurchsichtigen Strichen direkt aufs Canvas: wo zwei
187 // einander berühren, addiert Porter-Duff ihre Deckkraft – an Stufengrenzen,
188 // an den Kappen und überall, wo die Route sich selbst kreuzt.
189 const ROUTE_STUFEN = 20;
190 const ROUTE_MIN_ALPHA = 45;
191 const ROUTE_MAX_ALPHA = 255;
192
193 // Schweiflänge in Bildschirmpixeln, nicht in Wegpunkten: an Indizes gebunden
194 // hängt sie an der Abtastung des Pfads und fällt je Kapitel um Faktor 13
195 // verschieden aus. 400px lassen auch im kürzesten Kapitel (18, rund 770px

```

```

196 // Route) noch eine Hälfte auf der Mindestdeckkraft stehen.
197 const ROUTE_SCHWEIF_PX = 400;
198
199 let routenPuffer = null; // Vollbildpuffer, siehe zeichneRoute
200
201 // Zielwert der Stufe k (0 = älteste, ROUTE_STUFEN-1 = Spitze). Linear wie
202 // bisher; die Waschstärken unten leiten sich daraus ab.
203 function routenStufenAlpha(k) {
204     return ROUTE_MIN_ALPHA + (ROUTE_MAX_ALPHA - ROUTE_MIN_ALPHA) * k / (ROUTE_STUFEN - 1);
205 }
206
207 function routenPufferBereit() {
208     if (routenPuffer && (routenPuffer.width !== width || routenPuffer.height !== height)) {
209         routenPuffer.remove();
210         routenPuffer = null;
211     }
212     if (!routenPuffer) routenPuffer = createGraphics(width, height);
213     routenPuffer.clear();
214     return routenPuffer;
215 }
216
217 // Zerlegt den projizierten Pfad von der Spitze rückwärts in ROUTE_STUFEN Züge
218 // gleicher Bogenlänge; alles jenseits des Schweifs bildet die älteste Stufe.
219 // Grenzen werden ins Segment interpoliert, damit benachbarte Züge exakt
220 // aneinander anschliessen.
221 function routenStufenZuege(p, letzterPunkt) {
222     let abstand = new Array(letzterPunkt + 1);
223     abstand[letzterPunkt] = 0;
224     for (let i = letzterPunkt - 1; i >= 0; i--) {
225         abstand[i] = abstand[i + 1] + dist(p[i].x, p[i].y, p[i + 1].x, p[i + 1].y);
226     }
227     let stufenLaenge = ROUTE_SCHWEIF_PX / (ROUTE_STUFEN - 1);
228     let zuege = new Array(ROUTE_STUFEN);
229     let stufe = ROUTE_STUFEN - 1;
230     let zug = [p[letzterPunkt]];
231     for (let i = letzterPunkt - 1; i >= 0; i--) {
232         let grenze = (ROUTE_STUFEN - stufe) * stufenLaenge;
233         while (stufe > 0 && abstand[i] >= grenze) {
234             let segLaenge = abstand[i] - abstand[i + 1];
235             let t = segLaenge > 0 ? (grenze - abstand[i + 1]) / segLaenge : 0;
236             let gp = { x: lerp(p[i + 1].x, p[i].x, t), y: lerp(p[i + 1].y, p[i].y, t) };
237             zug.push(gp);
238             zuege[stufe] = zug;
239             stufe--;
240             zug = [gp];
241             grenze = (ROUTE_STUFEN - stufe) * stufenLaenge;
242         }
243         zug.push(p[i]);
244     }
245     zuege[stufe] = zug;
246     return zuege;
247 }
248
249 function zeichneRoute(punkte, upToIndex, bbox, strichstaerke = 2, offsetX = mapOffsetX, offsetY =
mapOffsetY, alphaMultiplifier = 1) {
250     if (upToIndex < 1 || alphaMultiplifier <= 0) return;
251     let letzterPunkt = Math.min(upToIndex, punkte.length - 1);
252     if (letzterPunkt < 1) return;
253
254     let p = [];
255     let links = Infinity, oben = Infinity, rechts = -Infinity, unten = -Infinity;
256     for (let i = 0; i <= letzterPunkt; i++) {
257         let q = lonLatToScreen(punkte[i][0], punkte[i][1], bbox, offsetX, offsetY);
258         p.push(q);
259         links = Math.min(links, q.x); rechts = Math.max(rechts, q.x);
260         oben = Math.min(oben, q.y); unten = Math.max(unten, q.y);

```

```

261 }
262 let rand = strichstaerke + 2;
263 let zuege = routenStufenZuege(p, letzterPunkt);
264
265 let pg = routenPufferBereit();
266 let schonGezeichnet = false;
267 for (let k = 0; k < ROUTE_STUFEN; k++) {
268   if (!zuege[k] || zuege[k].length < 2) continue;
269   // Kurze Routen fangen erst bei einer höheren Stufe an; bis dahin ist der
270   // Puffer leer und es gibt nichts zu waschen.
271   if (schonGezeichnet) {
272     // Nimmt allem bisher Gezeichneten den Anteil, der die vorige Stufe auf
273     // ihren Zielwert bringt. Nur über dem Routenrahmen statt Vollbild.
274
275     // ACHTUNG fill() muss VOR erase() stehen: erase() tauscht nur den
276     // Farbwert, es schaltet die Füllung nicht ein. Sonst malt das Rechteck
277     // nichts und der Verlauf bleibt ganz aus.
278     pg.push();
279     pg.noStroke();
280     pg.fill(255);
281     pg.erase(255 * (1 - routenStufenAlpha(k - 1) / routenStufenAlpha(k)));
282     pg.rect(links - rand, oben - rand, rechts - links + 2 * rand, unten - oben + 2 * rand);
283     pg.noErase();
284     pg.pop();
285   }
286   pg.push();
287   pg.noFill();
288   pg.strokeWeight(strichstaerke);
289   pg.strokeCap(ROUND);
290   pg.strokeJoin(ROUND);
291   pg.stroke(ROUTE_COLOR_RGB.r, ROUTE_COLOR_RGB.g, ROUTE_COLOR_RGB.b, ROUTE_MAX_ALPHA);
292   pg.beginPath();
293   zuege[k].forEach(q => pg.vertex(q.x, q.y));
294   pg.endShape();
295   pg.pop();
296   schonGezeichnet = true;
297 }
298
299 // alphaMultipller erst beim Auflegen, nicht im Puffer: so übersteht der
300 // Puffer die Einblendung eines Kapitels ohne Neuaufbau.
301 if (alphaMultipller < 1) tint(255, 255 * alphaMultipller);
302 image(pg, 0, 0);
303 if (alphaMultipller < 1) noTint();
304 }
305
306
307 // --- Export -----
308 // Drei Zeichenfunktionen. Leser: docs/architektur.md.
309 window.zeichneMassstabsleiste = zeichneMassstabsleiste;
310 window.zeichneWindrose = zeichneWindrose;
311 window.zeichneRoute = zeichneRoute;
312
313 }()); // Ende der Modulkapselung, siehe Kommentar oben

```